Hexera — Private x402 on Hedera

Nov 5, 2026 · @Dhananjay

Summary

Hexera makes x402 payments on Hedera unlinkable: the chain, the facilitator and the merchant can each see that a valid payment happened, but none of them can see who paid for what. It adds a new x402 payment scheme, `hexera-ecash`. Users deposit USDC once, then pay per request with blind-signed vouchers (RFC 9474). A public HCS topic acts as the double-spend registry, and merchants are paid in HTS batches.

Tagline: *Pay per request without publishing your reading history.*

Target tracks (3 picks): **Hedera x402** ($6.5k), **EF Best CROPS App** ($3.5k), **World Best Use of IDKit** ($2.5k).

The problem

Every x402 payment on Hedera publishes payer, merchant, amount and time on a public ledger. Hedera's own x402 docs list this as a limitation: no native privacy, with amounts, accounts and transfers visible on the public ledger (Hedera docs).

Per-request payments make this much worse than ordinary transfers, because each payment is one request. Anyone watching the chain can rebuild:

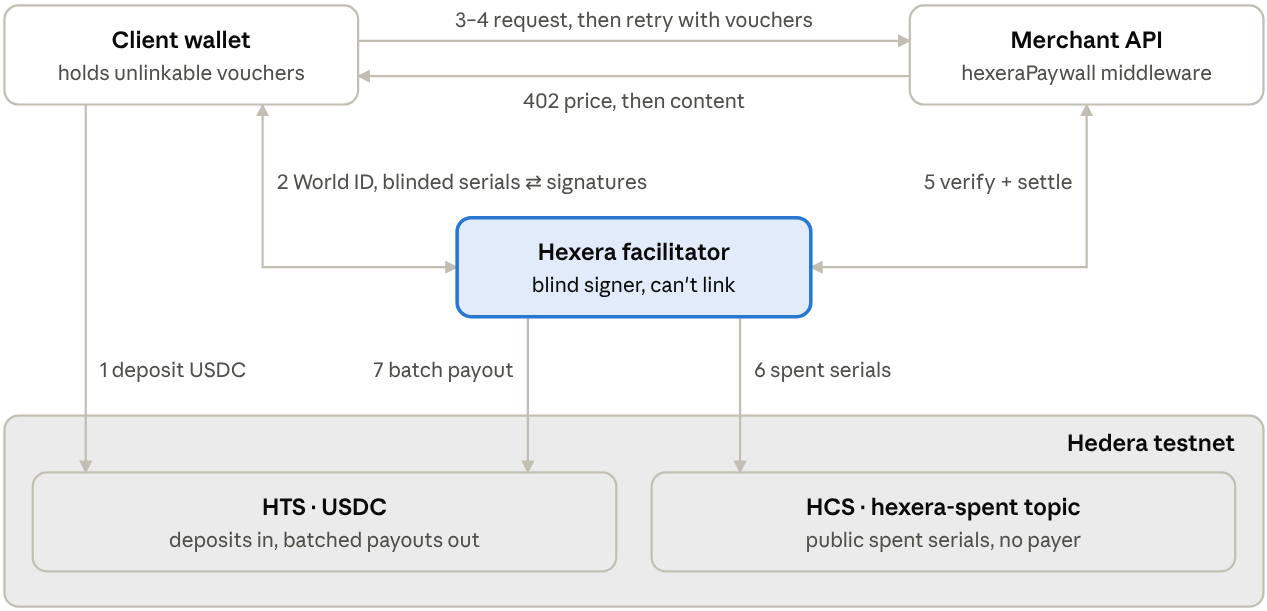
- **A reader's history.** Which articles, legal or health APIs, or datasets a person paid for, and when.
- **A company's strategy.** Which market-data feeds a trading desk buys, and how often it queries them.
- **A merchant's business.** Its full customer list and daily revenue, readable by competitors.

Card networks never expose this. The standard Hedera x402 scheme can't avoid it: the client signs a `TransferTransaction` that credits the merchant's account directly (Hedera exact scheme).

How it works

Money touches the chain only at deposit and payout. In between, payments are anonymous vouchers burned on a public HCS topic.

Payers and merchants meet only through anonymous vouchers



Hexera architecture · 7 steps

1. **Deposit.** The user verifies with World ID and deposits USDC (HTS) into the facilitator's threshold-key account.
2. **Get vouchers.** The client makes random serials, blinds them, and the facilitator signs them blind. The client unblinds them and stores the vouchers.
3. **Request.** The client calls a paywalled URL and gets `402` with `PAYMENT-REQUIRED` naming the `hexera-ecash` scheme and a price in units.
4. **Pay.** The client retries with N vouchers in `PAYMENT-SIGNATURE`, bound to this request's hash.
5. **Verify and settle.** The merchant asks the facilitator to verify the vouchers and settle them.
6. **Burn.** The facilitator writes the serials to the `hexera-spent` HCS topic. Consensus order decides double-spend races. The merchant serves the content.
7. **Payout.** Every few minutes, the facilitator pays each merchant in one HTS USDC transfer and posts the batch summary to `hexera-ledger`.

Protocol spec: the `hexera-ecash` scheme

`hexera-ecash` is an x402 v2 scheme: the server asks for N vouchers, the client sends them in `PAYMENT-SIGNATURE`, and the facilitator checks and burns them. It follows x402 v2's three base64-JSON headers: `PAYMENT-REQUIRED`, `PAYMENT-SIGNATURE`, `PAYMENT-RESPONSE` (x402 docs). x402 already has a `batch-settlement` scheme that confirms payment up front and redeems value on-chain later, so deferred settlement is not a foreign idea to the spec.

Voucher

- **Value:** one fixed unit, `U` = 0.001 USDC (1,000 base units at 6 decimals). Prices are whole numbers of units.
- **Serial:** 32 random bytes made by the client. Never shown to the facilitator before spending.
- **Signature:** RSA blind signature over the serial, using RSABSSA-SHA384-PSS-Randomized from `@cloudflare/blindrsa-ts` (npm).
- **Epoch key:** one RSA-2048 key pair per epoch (for example 24 h). Its public key and validity window are published on HCS. Vouchers from an old epoch must be spent or refunded within a grace period. Stretch: partially blind RSA, putting the epoch in public metadata so one key can serve many epochs.

On the wire a voucher is `{ "epoch": 12, "serial": "<hex>", "sig": "<base64>" }`.

Messages

1. **402 response.** `PAYMENT-REQUIRED` holds `accepts: [{ scheme: "hexera-ecash", network: "hedera:testnet", amount: "3" (units), asset: "<USDC token id>", payTo: "<merchant id in Hexera registry>", extra: { facilitator: "<url>", epoch: 12 } }]`. Servers can list `exact` beside it, so normal clients still work.
2. **Paid retry.** `PAYMENT-SIGNATURE` holds `{ x402Version: 2, accepted: {...}, payload: { vouchers: [v1, v2, v3], requestHash: "<sha256 of method+url+body>" } }`. The request hash stops a voucher set from being lifted onto another request.
3. **Settlement response.** `PAYMENT-RESPONSE` holds `{ success: true, network: "hedera:testnet", transactionId: "<HCS submit tx id>", serials: [...] }`. There is no `payer` field, on purpose.

Facilitator endpoints

Endpoint	Caller	Does
`POST /deposit/verify`	Client	Checks a USDC deposit tx on the mirror node and the World ID proof, then returns a deposit credit of K units
`POST /issue`	Client	Takes up to K blinded serials plus the deposit credit, returns K blind signatures, and marks the credit used
`POST /verify`	Merchant	Checks signatures, epoch and amount, and checks the HCS topic and local cache for already-spent serials
`POST /settle`	Merchant	Submits the serials to the HCS spent topic, waits for consensus, and credits the merchant's pending balance
`GET /supported`	Anyone	Lists schemes, network, current epoch public key and fees
`POST /payout` (cron)	Facilitator	Every few minutes, pays each merchant its pending balance in one HTS transfer and posts the batch summary to HCS

Double-spend rule: a serial is spent if it appears in the HCS spent topic. When two submissions race, the one with the earlier consensus timestamp wins and the later `/settle` fails.

Hedera services used

The whole system runs on native Hedera services through the JS SDK, with no Solidity. That keeps the "No Solidity Allowed" track open as a backup.

Service	Used for	Why it fits
HTS (USDC token)	Deposits into the facilitator account; batched payouts to merchants	Settles in a real stablecoin; one transfer can pay many merchants at once

Service	Used for	Why it fits
HCS topic `hexera-spent`	One message per payment: the spent serials and request hash	Consensus timestamps give a fair order for double-spend races; anyone can audit; each payment gets a Hedera transaction ID for HashScan
HCS topic `hexera-ledger`	Epoch public keys, deposit totals, payout batch summaries	Lets merchants and users check the facilitator's books
Accounts with a threshold key (KeyList)	The deposit account that holds user funds	Payouts need 2 of 3 keys, so one stolen server key can't drain deposits
Mirror Node REST API	Verifying deposits; reading the spent topic; building the audit page	Bonus point in the No Solidity track
Scheduled transactions (stretch)	The payout batch signed by the second key holder	Shows multi-sig without a contract

The facilitator pays all HCS and payout fees and recovers them as a small spread: each voucher is worth slightly less than it costs to buy (for example 1,000 vs 1,010 base units).

Trust and threat model

Privacy holds against everyone. Safety of funds relies on the facilitator, limited by a threshold key and public books. State this plainly in the README.

What each party learns

Party	Learns	Does not learn
Public chain observer	Account X deposited $Y to Hexera; merchant M received a payout of $Z; serials were spent at time T	Which depositor paid which merchant, or for which request
Facilitator	Who deposited and how much; which merchant redeemed which serials	Which depositor owns which serial (the signatures were blind)

Party	Learns	Does not learn
Merchant	A valid payment arrived, plus normal HTTP metadata	The payer's Hedera account or deposit
World ID	That a verification happened for the Hexera app	Anything about payments

The network layer is out of scope: the merchant still sees the client's IP address. The README should say so and point to Tor or a VPN.

Attacks and mitigations

Attack	Mitigation	Left over
Spend a voucher twice	HCS spent topic; earlier consensus timestamp wins	None
Reuse a voucher set on a different request	`requestHash` bound into the payload	None
Forge vouchers	RSA-2048 signatures; epoch keys	Key theft (keep the signing key off the payout server)
Facilitator issues unbacked vouchers	`hexera-ledger` publishes deposits and issued counts, so anyone can check issued ⩽ deposited	Needs someone to check
Facilitator runs off with deposits	2-of-3 threshold key; deposit cap per human	Trust in the key holders
Facilitator skips a merchant's payout	Merchant matches its own serials on HCS against payout batches	Merchant has to raise it
Facilitator links payer to spends by timing	Client delays first spends and pre-buys vouchers in the background	Small anonymity set at hackathon scale
Amount fingerprinting	Fixed denomination only	None

Attack	Mitigation	Left over
Facilitator gives each user a unique key to tag them	One epoch key, published on HCS; client refuses any key not on HCS	None
Money laundering through Hexera	No cash-out to arbitrary accounts; per-human deposit cap via World ID	Merchant collusion (registered merchants only)

Design decisions

Each choice below was tested against the obvious objection. Judges will ask the same questions.

Why blind signatures and not a zk shielded pool? A shielded pool needs a proof and an on-chain verification for every payment, which costs more time and fees than a $0.001 call. Blind RSA costs a few milliseconds per voucher, needs no circuits, and has a mature standard (RFC 9474) with Cloudflare's library. **Trade-off:** the facilitator holds deposits. That's covered by the threshold key and public books.

Why one shared facilitator and not merchant-issued vouchers? If each merchant issued its own vouchers, the deposit itself would show "account X prepaid merchant M", which leaks the relationship. One facilitator for many merchants means a deposit only shows "X uses Hexera".

Isn't it a mixer? It's built not to be one. Vouchers can only leave the system as payment to a registered merchant, never as a withdrawal to a fresh account. Deposits are capped per human (for example $20 a day). Accounts are free to create, so a per-account cap means nothing; that's why the cap is tied to World ID.

Why World ID's Proof of Human, not Passport or Selfie Check? The cap only needs to know "one unique human", not age, nationality or face match. Proof of Human is the minimum that does the job, and its per-app nullifier reveals nothing else. That's the exact reasoning the IDKit track asks you to explain.

What happens to unspent vouchers? No cash-out. At the end of an epoch, the client trades old vouchers for new-epoch ones: it burns the old serials on HCS and gets the same number of fresh blind signatures. Value carries over and stays unlinkable.

Why not settle every payment on-chain with HTS? That would be the normal `exact` scheme, which is public. HCS gives a per-payment Hedera transaction and ordering for double-spend checks, without revealing the payer. Money moves on HTS only at deposit and payout time.

Doesn't timing still link a deposit to the first spend? Yes, it can. The client wallet buys vouchers in the background ahead of time and adds random delays. The README admits that privacy grows with the number of users.

Who actually needs this? Readers paying per article, people using health or legal APIs, companies buying data feeds, and merchants who don't want their customer list public. None of these need AI; any HTTP client can pay.

Tracks and requirement checklist

Three partner picks: Hedera, EF and World. Requirements are copied from the prize list; tick them off as you go.

Hedera — AI Agents That Pay: x402 on Hedera ($6.5k)

The track says "agent or client", so a plain client counts. Its ideas list "micropaywalls for creators" and "an open-source Hedera x402 facilitator with its own safety checks and replay protection", which is exactly this.

- [ ] At least one x402-protected endpoint settling on Hedera testnet, through a facilitator you run yourself
- [ ] A client that finds the endpoint and completes a paid request end to end with no human approving
- [ ] Public repo, README with setup, architecture, payment flow and at least one Hedera transaction ID of a settled payment
- [ ] Demo video ⩽ 5 min showing a payment settling, linked on HashScan
- [ ] Extra points: HCS tamper-evident audit log, USDC on HTS, spending controls (per-human cap)

Judging risk: a judge may say the HCS message isn't a "settled payment". Show both in the README: the per-request HCS transaction and the HTS payout transaction it rolled into. Optionally also accept the standard `exact` scheme on the same endpoint, to show Hexera works beside it.

Backup if allowed by the rules: "No Solidity Allowed" ($2k), which needs the JS SDK only, at least two native services (HTS + HCS) and a demo ⩽ 5 min.

EF — Best CROPS App ($3.5k)

The track lists "a payment, messaging or social app where your server learns nothing about who talks to whom". Lead property: **P (private)**.

- [ ] Public repo under MIT or Apache-2.0, README explains how to run it
- [ ] README section titled **CROPS**, one or two lines per letter (draft below)
- [ ] Demo video ⩽ 3 min showing the lead property at the moment it matters: the facilitator and merchant logs, with nothing linking payer to merchant
- [ ] No third-party trackers or analytics in the client; keys stay on the user's device

CROPS draft for the README:

- **C:** The facilitator is the one party users depend on. If it refuses someone, they can't get vouchers. Anyone can run their own facilitator from the repo, and merchants can accept several.
- **O:** MIT. Facilitator, client and merchant middleware all build from the repo alone.
- **P:** The facilitator sees deposits and redemptions but can't link them. Merchants see no account. The chain sees deposits, anonymous serials and payouts. IP addresses are not hidden.
- **S:** Voucher secrets stay in the client. Deposits sit behind a 2-of-3 threshold key. The trust assumptions are listed in the threat model.

World — Best Use of IDKit ($2.5k)

- [ ] IDKit in a working app, Proof of Human verified on the server
- [ ] Explain the trust moment: a deposit above the free limit needs one unique human, and Proof of Human is the minimum that proves it
- [ ] Show success and one alternative path: a second account controlled by the same human is refused past the cap
- [ ] Integration debrief: time to first success, friction, missing docs, the one biggest improvement

Repo structure and stack

A pnpm TypeScript monorepo with five packages and one demo app. Commit steadily from the first hour; several tracks check commit history.

```
hexera/
  packages/
    core/          voucher types, encoding, blind-RSA wrapper, requestHash
    facilitator/   Fastify server: /deposit/verify /issue /verify /settle
/supported, payout cron
    client/        wallet: deposit, blind + store vouchers (IndexedDB), fetch
wrapper for 402
    server/        Express middleware: hexeraPaywall({ price: 3 }) for any
route
    hedera/        HTS deposit/payout, HCS topics, mirror-node reads,
threshold-key account setup
  apps/
    reader/        Next.js demo: two "publishers", paywalled articles, wallet
panel
    audit/         one page: reads hexera-spent + hexera-ledger from the
mirror node
  scripts/         create-topics.ts, create-token-association.ts, seed-
merchants.ts
  README.md  CROPS section  WORLD_DEBRIEF.md
```

Piece	Choice
Hedera	`@hiero-ledger/sdk` (JS SDK), mirror node REST
Blind signatures	`@cloudflare/blindrsa-ts`, RSABSSA-SHA384-PSS-Randomized, 2048-bit
x402 types	`@x402/core` for headers and payload types; `hexera-ecash` registered as a custom scheme
World ID	IDKit (web widget) + server-side proof verification
Storage	Facilitator: SQLite for the spent-serial cache and merchant balances (HCS stays the source of truth). Client: IndexedDB
Optional Rust	Swap the blind signer for a Rust service if you want that part in Rust; not needed for the demo

Note: Hedera's docs say `@x402/hedera` (the `exact` scheme) may need to be used from the x402 monorepo source rather than npm (Hedera docs). Hexera doesn't depend on it, only on `@x402/core`.

Build plan (about 36 hours)

Get the riskiest piece working first: one voucher bought, spent and burned on HCS by hour 8. Everything after that is wiring and the demo.

Before the event

- [ ] Hedera testnet accounts (facilitator, 2 merchants, 2 users), testnet USDC token id, token associations
- [ ] Run `@cloudflare/blindrsa-ts` blind → sign → finalize → verify in a Node script
- [ ] Read the x402 v2 spec headers and the World IDKit quickstart

Hours 0–8: core loop

- [ ] `core`: voucher type, encode/decode, requestHash
- [ ] `hedera`: create `hexera-spent` and `hexera-ledger` topics; submit and read messages via the mirror node
- [ ] `facilitator`: `/issue` (no deposit check yet), `/verify`, `/settle` writing serials to HCS
- [ ] Script: issue 3 vouchers, spend 1, try to spend it again and see it refused

Hours 8–16: real money and the paywall

- [ ] `/deposit/verify`: check the HTS USDC deposit on the mirror node, return a deposit credit
- [ ] `server` middleware: 402 with `PAYMENT-REQUIRED`, verify + settle on retry, `PAYMENT-RESPONSE`
- [ ] `client` fetch wrapper: on 402, pick N vouchers and retry
- [ ] Payout cron: one HTS transfer per merchant, batch summary on `hexera-ledger`

Hours 16–24: demo app and World ID

- [ ] `reader` app: two publishers, paywalled articles, wallet panel showing the voucher balance

- [ ] IDKit Proof of Human at deposit; server-side proof check; per-human daily cap
- [ ] Threshold-key deposit account (2 of 3) for payouts

Hours 24–30: audit page and hardening

- [ ] `audit` app: spent serials, deposits vs issued, payout batches, all from the mirror node
- [ ] Epoch key published on HCS; client refuses unknown keys
- [ ] Background voucher top-up and random spend delays in the client

Hours 30–36: submission

- [ ] README: setup, architecture, payment flow, HashScan links, CROPS section, threat model, limitations
- [ ] WORLD_DEBRIEF.md
- [ ] Record the demo (≤ 3 min for EF; the same video works for Hedera and World)
- [ ] Cut list if late: threshold key → single key (say so in README); audit page → HashScan links; epoch rotation → one fixed epoch

Demo script (3 minutes)

The point to land: two publishers got paid, and nobody can tell who paid them.

1. **0:00 — The problem (20 s).** Show a normal x402 payment on HashScan: payer account → merchant, readable by anyone.
2. **0:20 — Deposit (30 s).** Verify with World ID, deposit $2 USDC, and the wallet shows 2,000 vouchers. HashScan shows only "account deposited to Hexera".
3. **0:50 — Read (40 s).** Open 5 paywalled articles across 2 publishers. Each loads after an automatic 402 → pay → 200. No popups.
4. **1:30 — What the chain sees (30 s).** HashScan and the audit page: anonymous serials on `hexera-spent`, then a payout batch to each publisher. Nothing links the reader to either one.
5. **2:00 — What the servers see (20 s).** Facilitator and publisher logs side by side: valid payments, no account. This is the CROPS moment.
6. **2:20 — Attacks fail (25 s).** Replay a voucher: refused, the serial is already on HCS. A second account of the same human tries to deposit past the cap: World ID refuses it.
7. **2:45 — Close (15 s).** "x402 with card-network privacy, on Hedera, no smart contracts, open source."

Risks and open questions

The biggest risk is judging, not code: confirm with the Hedera booth on day one that HCS-per-request plus HTS payout counts as x402 settlement.

Risk	Fallback
Hedera judges don't accept deferred settlement	Also accept `exact` on the same endpoint; show both in the demo
Testnet USDC token is hard to get or associate	Make your own HTS stablecoin for testing, and say so; ask the booth for the official test token
IDKit setup takes longer than planned	Do it at hour 16 at the latest; if blocked, keep a per-account cap and drop the World pick
Blind-RSA in the browser is slow	Blind in a Web Worker; issue in batches of 100
Anonymity set of 1–2 users in the demo	Say so; show the design scales with users

- [ ] Can one project place in both Hedera x402 and No Solidity? Check the ETHGlobal rules.
- [ ] Is a $20/day per-human cap sensible, or should it be lower?
- [ ] Voucher unit: is $0.001 right for article paywalls, or $0.01?

After Mumbai

Propose `hexera-ecash` as a scheme to the x402 repo, add support for several facilitators per merchant, and look at a zk version that removes trust in the facilitator for deposits.

Sources

- Hedera: x402 overview and limitations
- Hedera: x402 exact scheme
- Hedera: x402 in AI Studio, facilitators and packages
- x402: HTTP 402 and v2 headers
- x402: v1 to v2 migration guide
- @cloudflare/blindrsa-ts (RFC 9474)

- ETHGlobal Mumbai prize list (the JSON you shared)